HDOC Day-48

Write a C program that reads the size and elements of a variable-length integer array, arr[]. Pass the array to a function named pivot_index() using its address. Use pointers to pass the array between the calling function and pivot_index(). The function pivot_index() must:

- Calculate the left sum (sum of all elements strictly to the left of each index).
- Calculate the right sum (sum of all elements strictly to the right of each index).
- Treat the left sum = 0 for the first element.
- Treat the right sum = 0 for the last element.
- Find and display the leftmost index at which left sum = right sum.
- Display -1 if no pivot index exists.

Example 1:

Array: 1 7 3 6 5 6

Output:

Pivot index: 3 [Explanation: Index 3 is a pivot index because the left sum (1+7+3) is equal to the right sum (5+6)]

Example 2:

Array: 1 2 3

Output:

-1 [Explanation: There is no pivot index, as no left sum can be found to be equal to a right sum]

Example 3:

Array: 2 1 -1

Output:

Pivot index: 0 [Explanation: Index 0 is a pivot index because the left sum (0) is equal to the right sum (1-1)]

1. Algorithm

1. Start.
2. Read the size n of the array.
3. Declare an integer array arr[n] and read its elements.
4. Calculate the total sum of all array elements.
5. Pass the array address and its size to the function pivot_index().
6. Initialize leftSum = 0.
7. Traverse the array from left to right:
    - Calculate rightSum = totalSum - leftSum - arr[i].
    - If leftSum == rightSum, display the index i and stop searching.
    - Otherwise, add arr[i] to leftSum.
8. If no pivot index is found, display -1.
9. Stop.

2. C program

```
#include <stdio.h>

int pivot_index(int *arr, int n)

{

  int totalSum = 0;

  int leftSum = 0;

  int rightSum;


  // Calculate total sum

  for (int i = 0; i < n; i++)

  {

    totalSum += *(arr + i);

  }


  // Find the leftmost pivot index
```

```
    for (int i = 0; i < n; i++)
    {
        rightSum = totalSum - leftSum - *(arr + i);

        if (leftSum == rightSum)
        {
            return i;
        }

        leftSum += *(arr + i);
    }

    return -1;
}

int main()
{
    int n;

    printf("Enter the size of the array: ");
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("Invalid array size.\n");
        return 0;
    }
```

```
    int arr[n];

    printf("Enter the array elements: ");
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    int result = pivot_index(arr, n);

    if (result != -1)
        printf("Pivot index: %d\n", result);
    else
        printf("-1\n");
    return 0;
}
```

Test Case Table:

Test Case	Input array	Expected Output	Explanation
1	1 7 3 6 5 6	Pivot index: 3	Left sum = 11, right sum = 11
2	1 2 3	-1	No index has equal left and right sums
3	2 1 -1	Pivot index: 0	Left sum = 0, right sum = 0
4	1	Pivot index: 0	Both sums are 0
5	1 2 3 4 6	Pivot index: 3	Left sum = 6 right sum = 6

Compilation and Execution:

The following program has been compiled and executed successfully.

Test Case 1:

```
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter the size of the array: 6
Enter the array elements: 1 7 3 6 5 6
Pivot index: 3
```

Test Case 2:

```
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter the size of the array: 3
Enter the array elements: 1 2 3
-1
```

Test Case 3:

```
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter the size of the array: 3
Enter the array elements: 2 1 -1
Pivot index: 0
```

Test Case 4:

```
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter the size of the array: 1
Enter the array elements: 1
Pivot index: 0
```

Test Case 5:

```
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter the size of the array: 5
Enter the array elements: 1 2 3 4 6
Pivot index: 3
```